DBS - JOINS AND SET OPERATIONS

PART 1: CROSS JOIN

STEP 1: CREATE TABLES

```
CREATE TABLE class (
    id INT,
    name VARCHAR(30)
);

CREATE TABLE class_info (
    id INT,
    address VARCHAR(30)
);
```

```
mysql> USE sec13;
Database changed
mysql> DROP TABLE IF EXISTS class;
Query OK, 0 rows affected (0.06 sec)

mysql> DROP TABLE IF EXISTS class_info;
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE class (
    ->     id INT,
    ->     name VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE class_info (
    ->     id INT,
    ->     address VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO class VALUES
    -> (1,'Ananya'),
    -> (2,'Akhil'),
    -> (4,'Bala');
Query OK, 3 rows affected (0.03 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO class_info VALUES
    -> (1,'DELHI'),
    -> (2,'MUMBAI'),
    -> (3,'CHENNAI');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT *
    -> FROM class
    -> CROSS JOIN class_info;
+------+--------+------+---------+
| id   | name   | id   | address |
+------+--------+------+---------+
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    1 | DELHI   |
|    4 | Bala   |    1 | DELHI   |
|    1 | Ananya |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
|    4 | Bala   |    2 | MUMBAI  |
|    1 | Ananya |    3 | CHENNAI |
|    2 | Akhil  |    3 | CHENNAI |
|    4 | Bala   |    3 | CHENNAI |
+------+--------+------+---------+
9 rows in set (0.01 sec)
```

STEP 2: INSERT DATA

```
INSERT INTO class VALUES
(1,'Ananya'),
(2,'Akhil'),
(4,'Bala');

INSERT INTO class_info VALUES
(1,'DELHI'),
(2,'MUMBAI'),
(3,'CHENNAI');
```

```
mysql> INSERT INTO class VALUES
    -> (1,'Ananya'),
    -> (2,'Akhil'),
    -> (4,'Bala');
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO class_info VALUES
    -> (1,'DELHI'),
    -> (2,'MUMBAI'),
    -> (3,'CHENNAI');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM class;
+------+--------+
| id   | name   |
+------+--------+
|    1 | Ananya |
|    2 | Akhil  |
|    4 | Bala   |
|    1 | Ananya |
|    2 | Akhil  |
|    4 | Bala   |
+------+--------+
6 rows in set (0.00 sec)

mysql> SELECT * FROM class_info;
+------+---------+
| id   | address |
+------+---------+
|    1 | DELHI   |
|    2 | MUMBAI  |
|    3 | CHENNAI |
|    1 | DELHI   |
|    2 | MUMBAI  |
|    3 | CHENNAI |
+------+---------+
6 rows in set (0.00 sec)
```

```
mysql> SELECT *
    -> FROM class
    -> CROSS JOIN class_info;
+------+--------+------+---------+
| id   | name   | id   | address |
+------+--------+------+---------+
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    1 | DELHI   |
|    4 | Bala   |    1 | DELHI   |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    1 | DELHI   |
|    4 | Bala   |    1 | DELHI   |
|    1 | Ananya |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
|    4 | Bala   |    2 | MUMBAI  |
|    1 | Ananya |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
|    4 | Bala   |    2 | MUMBAI  |
|    1 | Ananya |    3 | CHENNAI |
|    2 | Akhil  |    3 | CHENNAI |
|    4 | Bala   |    3 | CHENNAI |
|    1 | Ananya |    3 | CHENNAI |
|    2 | Akhil  |    3 | CHENNAI |
|    4 | Bala   |    3 | CHENNAI |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    1 | DELHI   |
|    4 | Bala   |    1 | DELHI   |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    1 | DELHI   |
|    4 | Bala   |    1 | DELHI   |
|    1 | Ananya |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
|    4 | Bala   |    2 | MUMBAI  |
|    1 | Ananya |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
|    4 | Bala   |    2 | MUMBAI  |
|    1 | Ananya |    3 | CHENNAI |
|    2 | Akhil  |    3 | CHENNAI |
|    4 | Bala   |    3 | CHENNAI |
|    1 | Ananya |    3 | CHENNAI |
|    2 | Akhil  |    3 | CHENNAI |
+------+--------+------+---------+
36 rows in set (0.00 sec)
```

QUESTION 1 - CROSS JOIN

SCENARIO:
School administration wants every student to be paired with every available city for survey analysis.

QUERY:

```
SELECT *
FROM class
CROSS JOIN class_info;
```

```
mysql> USE sec13;
Database changed
mysql> SELECT * FROM class;
+------+--------+
| id   | name   |
+------+--------+
|    1 | Ananya |
|    2 | Akhil  |
|    4 | Bala   |
|    1 | Ananya |
|    2 | Akhil  |
|    4 | Bala   |
+------+--------+
6 rows in set (0.00 sec)

mysql> SELECT * FROM class_info;
+------+---------+
| id   | address |
+------+---------+
|    1 | DELHI   |
|    2 | MUMBAI  |
|    3 | CHENNAI |
|    1 | DELHI   |
|    2 | MUMBAI  |
|    3 | CHENNAI |
+------+---------+
6 rows in set (0.00 sec)
```

```
mysql> SELECT *
    -> FROM class
    -> CROSS JOIN class_info;
+------+--------+------+---------+
| id   | name   | id   | address |
+------+--------+------+---------+
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    1 | DELHI   |
|    4 | Bala   |    1 | DELHI   |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    1 | DELHI   |
|    4 | Bala   |    1 | DELHI   |
|    1 | Ananya |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
|    4 | Bala   |    2 | MUMBAI  |
|    1 | Ananya |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
|    4 | Bala   |    2 | MUMBAI  |
|    1 | Ananya |    3 | CHENNAI |
|    2 | Akhil  |    3 | CHENNAI |
|    4 | Bala   |    3 | CHENNAI |
|    1 | Ananya |    3 | CHENNAI |
|    2 | Akhil  |    3 | CHENNAI |
|    4 | Bala   |    3 | CHENNAI |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    1 | DELHI   |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    1 | DELHI   |
|    4 | Bala   |    1 | DELHI   |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    1 | DELHI   |
|    4 | Bala   |    1 | DELHI   |
|    1 | Ananya |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
|    4 | Bala   |    2 | MUMBAI  |
|    1 | Ananya |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
|    4 | Bala   |    2 | MUMBAI  |
|    1 | Ananya |    3 | CHENNAI |
|    2 | Akhil  |    3 | CHENNAI |
|    4 | Bala   |    3 | CHENNAI |
|    1 | Ananya |    3 | CHENNAI |
|    2 | Akhil  |    3 | CHENNAI |
|    4 | Bala   |    3 | CHENNAI |
+------+--------+------+---------+
36 rows in set (0.00 sec)
```

PART 2: INNER JOIN

```
CREATE TABLE class (
    id INT,
    name VARCHAR(30)
);

CREATE TABLE class_info (
    id INT,
    address VARCHAR(30)
);

INSERT INTO class VALUES
(1,'ananya'),
(2,'akhil'),
(3,'bala'),
(4,'anu');

INSERT INTO class_info VALUES
(1,'DELHI'),
(2,'MUMBAI'),
(3,'CHENNAI');
```

```
mysql> SELECT *
    -> FROM class
    -> INNER JOIN class_info
    -> ON class.id = class_info.id;
+------+--------+------+---------+
| id   | name   | id   | address |
+------+--------+------+---------+
|    1 | Ananya |    1 | DELHI   |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
|    1 | Ananya |    1 | DELHI   |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
+------+--------+------+---------+
8 rows in set (0.01 sec)
```

QUESTION 2 - INNER JOIN

SCENARIO:
Management wants only students whose address details are available in both tables.

QUERY:

```
SELECT *
FROM class
INNER JOIN class_info
ON class.id = class_info.id;
```

```
mysql> SELECT *
    -> FROM class
    -> INNER JOIN class_info
    -> ON class.id = class_info.id;
+------+--------+------+---------+
| id   | name   | id   | address |
+------+--------+------+---------+
|    1 | Ananya |    1 | DELHI   |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
|    1 | Ananya |    1 | DELHI   |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
+------+--------+------+---------+
8 rows in set (0.01 sec)
```

QUESTION 3 - INNER JOIN

SCENARIO:
Find student names and addresses for registered students.

QUERY:

```
SELECT class.name,
       class_info.address
FROM class
INNER JOIN class_info
ON class.id=class_info.id;
```

```
mysql> SELECT class.name,
    ->        class_info.address
    -> FROM class
    -> INNER JOIN class_info
    -> ON class.id = class_info.id;
+--------+---------+
| name   | address |
+--------+---------+
| Ananya | DELHI   |
| Ananya | DELHI   |
| Akhil  | MUMBAI  |
| Akhil  | MUMBAI  |
| Ananya | DELHI   |
| Ananya | DELHI   |
| Akhil  | MUMBAI  |
| Akhil  | MUMBAI  |
+--------+---------+
8 rows in set (0.00 sec)
```

PART 3: NATURAL JOIN

QUESTION 4 - NATURAL JOIN

SCENARIO:
The database administrator wants matching records automatically joined using common columns.

QUERY:

```
SELECT *
FROM class
NATURAL JOIN class info;
```

```
mysql> SELECT *
    -> FROM class
    -> NATURAL JOIN class_info;
+------+--------+---------+
| id   | name   | address |
+------+--------+---------+
|    1 | Ananya | DELHI   |
|    1 | Ananya | DELHI   |
|    2 | Akhil  | MUMBAI  |
|    2 | Akhil  | MUMBAI  |
|    1 | Ananya | DELHI   |
|    1 | Ananya | DELHI   |
|    2 | Akhil  | MUMBAI  |
|    2 | Akhil  | MUMBAI  |
+------+--------+---------+
8 rows in set (0.02 sec)
```

PART 4: LEFT OUTER JOIN

```
INSERT INTO class VALUES
(5,'ashish');

INSERT INTO class_info VALUES
(7,'NOIDA'),
(8,'PANIPAT');
```

```
mysql> INSERT INTO class VALUES
    -> (5,'ashish');
Query OK, 1 row affected (0.04 sec)

mysql> INSERT INTO class_info VALUES
    -> (7,'NOIDA'),
    -> (8,'PANIPAT');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

QUESTION 5 - LEFT JOIN

SCENARIO:
School wants all students even if address information is not available.

QUERY:

```
SELECT *
FROM class
LEFT OUTER JOIN class_info
ON class.id=class_info.id;
```

```
mysql> SELECT *
    -> FROM class
    -> LEFT OUTER JOIN class_info
    -> ON class.id = class_info.id;
+------+--------+------+---------+
| id   | name   | id   | address |
+------+--------+------+---------+
|    1 | Ananya |    1 | DELHI   |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
|    4 | Bala   | NULL | NULL    |
|    1 | Ananya |    1 | DELHI   |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
|    4 | Bala   | NULL | NULL    |
|    5 | ashish | NULL | NULL    |
+------+--------+------+---------+
11 rows in set (0.01 sec)
```

QUESTION 6 - LEFT JOIN

SCENARIO:
Find students who do not have address records.

QUERY:

```
SELECT *
FROM class
LEFT JOIN class_info
ON class.id=class_info.id
WHERE class_info.id IS NULL;
```

```
mysql> SELECT *
    -> FROM class
    -> LEFT JOIN class_info
    -> ON class.id = class_info.id
    -> WHERE class_info.id IS NULL;
+------+--------+------+---------+
| id   | name   | id   | address |
+------+--------+------+---------+
|    4 | Bala   | NULL | NULL    |
|    4 | Bala   | NULL | NULL    |
|    5 | ashish | NULL | NULL    |
+------+--------+------+---------+
3 rows in set (0.01 sec)
```

\

PART 5: RIGHT OUTER JOIN

QUESTION 7 - RIGHT JOIN

SCENARIO:
Administration wants all address records even if students are missing.

QUERY:

```
SELECT *
FROM class
RIGHT OUTER JOIN class_info
ON class.id=class_info.id;
```

```
mysql> SELECT *
    -> FROM class
    -> RIGHT OUTER JOIN class_info
    -> ON class.id=class_info.id;
+------+--------+------+---------+
| id   | name   | id   | address |
+------+--------+------+---------+
|    1 | Ananya |    1 | DELHI   |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
| NULL | NULL   |    3 | CHENNAI |
|    1 | Ananya |    1 | DELHI   |
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    2 | MUMBAI  |
|    2 | Akhil  |    2 | MUMBAI  |
| NULL | NULL   |    3 | CHENNAI |
| NULL | NULL   |    7 | NOIDA   |
| NULL | NULL   |    8 | PANIPAT |
+------+--------+------+---------+
12 rows in set (0.00 sec)
```

QUESTION 8 - RIGHT JOIN

SCENARIO:
Find address records that do not belong to any student.

QUERY:

```
SELECT *
FROM class
RIGHT JOIN class_info
ON class.id=class_info.id
WHERE class.id IS NULL;
```

```
mysql> SELECT *
    -> FROM class
    -> RIGHT JOIN class_info
    -> ON class.id=class_info.id
    -> WHERE class.id IS NULL;
+------+------+------+---------+
| id   | name | id   | address |
+------+------+------+---------+
| NULL | NULL |    3 | CHENNAI |
| NULL | NULL |    3 | CHENNAI |
| NULL | NULL |    7 | NOIDA   |
| NULL | NULL |    8 | PANIPAT |
+------+------+------+---------+
4 rows in set (0.00 sec)
```

PART 6: FULL OUTER JOIN

QUESTION 9 - FULL OUTER JOIN

SCENARIO:
Management wants all records from both tables, matched and unmatched.

QUERY:

```
SELECT *
FROM class
FULL OUTER JOIN class_info
ON class.id=class_info.id;
```

```
mysql> SELECT *
    -> FROM class
    -> LEFT JOIN class_info
    -> ON class.id = class_info.id
    ->
    -> UNION
    ->
    -> SELECT *
    -> FROM class
    -> RIGHT JOIN class_info
    -> ON class.id = class_info.id;
+------+--------+------+---------+
| id   | name   | id   | address |
+------+--------+------+---------+
|    1 | Ananya |    1 | DELHI   |
|    2 | Akhil  |    2 | MUMBAI  |
|    4 | Bala   | NULL | NULL    |
|    5 | ashish | NULL | NULL    |
| NULL | NULL   |    3 | CHENNAI |
| NULL | NULL   |    7 | NOIDA   |
| NULL | NULL   |    8 | PANIPAT |
+------+--------+------+---------+
7 rows in set (0.04 sec)
```

QUESTION 10 - FULL OUTER JOIN

SCENARIO:
Audit team wants unmatched records from both tables.

QUERY:

```
SELECT *
FROM class
FULL OUTER JOIN class_info
ON class.id=class_info.id
WHERE class.id IS NULL
OR class_info.id IS NULL;
```

```
mysql> SELECT *
    -> FROM class
    -> LEFT JOIN class_info
    -> ON class.id = class_info.id
    -> WHERE class_info.id IS NULL
    ->
    -> UNION
    ->
    -> SELECT *
    -> FROM class
    -> RIGHT JOIN class_info
    -> ON class.id = class_info.id
    -> WHERE class.id IS NULL;
+------+--------+------+---------+
| id   | name   | id   | address |
+------+--------+------+---------+
|    4 | Bala   | NULL | NULL    |
|    5 | ashish | NULL | NULL    |
| NULL | NULL   |    3 | CHENNAI |
| NULL | NULL   |    7 | NOIDA   |
| NULL | NULL   |    8 | PANIPAT |
+------+--------+------+---------+
5 rows in set (0.00 sec)
```

PART 7: UNION

```
CREATE TABLE first_table(
id INT,
name VARCHAR(30)
);

CREATE TABLE second_table(
id INT,
name VARCHAR(30)
);
```

```
INSERT INTO first_table VALUES
(1,'abhi'),
(2,'adam');

INSERT INTO second_table VALUES
(2,'adam'),
(3,'chester');
```

```
mysql> CREATE TABLE first_table (
    ->     id INT,
    ->     name VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.16 sec)

mysql> CREATE TABLE second_table (
    ->     id INT,
    ->     name VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.08 sec)

mysql> INSERT INTO first_table VALUES
    -> (1,'abhi'),
    -> (2,'adam');
Query OK, 2 rows affected (0.03 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> INSERT INTO second_table VALUES
    -> (2,'adam'),
    -> (3,'chester');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM first_table;
+------+------+
| id   | name |
+------+------+
|    1 | abhi |
|    2 | adam |
+------+------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM second_table;
+------+---------+
| id   | name    |
+------+---------+
|    2 | adam    |
|    3 | chester |
+------+---------+
```

```
mysql> SELECT * FROM second_table;
+------+---------+
| id   | name    |
+------+---------+
|    2 | adam    |
|    3 | chester |
+------+---------+
2 rows in set (0.00 sec)
```

QUESTION 11 - UNION

SCENARIO:
Management wants a combined list without duplicates.

QUERY:

```
SELECT * FROM first_table
UNION
SELECT * FROM second_table;
```

```
mysql> SELECT * FROM first_table
    -> UNION
    -> SELECT * FROM second_table;
+------+---------+
| id   | name    |
+------+---------+
|    1 | abhi    |
|    2 | adam    |
|    3 | chester |
+------+---------+
3 rows in set (0.00 sec)
```

QUESTION 12 - UNION

SCENARIO:
Generate a master student list from two branches.

QUERY:

```
SELECT name FROM first_table
UNION
SELECT name FROM second_table;
```

```
mysql> SELECT name FROM first_table
    -> UNION
    -> SELECT name FROM second_table;
+---------+
| name    |
+---------+
| abhi    |
| adam    |
| chester |
+---------+
3 rows in set (0.00 sec)
```

PART 8: UNION ALL

QUESTION 13 - UNION ALL

SCENARIO:
Management wants all records including duplicates.

QUERY:

```
SELECT * FROM first_table
UNION ALL
SELECT * FROM second_table;
```

```
mysql> SELECT * FROM first_table
    -> UNION ALL
    -> SELECT * FROM second_table;
+------+---------+
| id   | name    |
+------+---------+
|    1 | abhi    |
|    2 | adam    |
|    2 | adam    |
|    3 | chester |
+------+---------+
4 rows in set (0.00 sec)
```

QUESTION 14 - UNION ALL

SCENARIO:
Count total records from both branches.

QUERY:

```
SELECT COUNT(*)
FROM
(
SELECT * FROM first_table
```

```
UNION ALL
SELECT * FROM second_table
) A;
```

```
mysql> SELECT COUNT(*)
    -> FROM
    -> (
    -> SELECT * FROM first_table
    -> UNION ALL
    -> SELECT * FROM second_table
    -> ) A;
+----------+
| COUNT(*) |
+----------+
|        4 |
+----------+
1 row in set (0.02 sec)
```

PART 9: INTERSECT

QUESTION 15 - INTERSECT

SCENARIO:
Find students present in both branches.

QUERY:

```
SELECT * FROM first_table
INTERSECT
SELECT * FROM second_table;
```

```
mysql> SELECT *
    -> FROM first_table
    -> WHERE (id, name) IN (
    ->     SELECT id, name
    ->     FROM second_table
    -> );
+------+------+
| id   | name |
+------+------+
|    2 | adam |
+------+------+
1 row in set (0.01 sec)
```

QUESTION 16 - INTERSECT

SCENARIO:
Find common student names.

QUERY:

```
SELECT name FROM first_table
INTERSECT
SELECT name FROM second_table;
```

```
mysql> SELECT name
    -> FROM first_table
    -> WHERE name IN (
    ->     SELECT name
    ->     FROM second_table
    -> );
+------+
| name |
+------+
| adam |
+------+
1 row in set (0.00 sec)
```

PART 10: MINUS

QUESTION 17 - MINUS

SCENARIO:
Find students present only in the first branch.

QUERY:

```
SELECT * FROM first_table
MINUS
SELECT * FROM second_table;
```

```
mysql> SELECT *
    -> FROM first_table
    -> WHERE NOT EXISTS (
    ->     SELECT 1
    ->     FROM second_table
    ->     WHERE second_table.id = first_table.id
    ->       AND second_table.name = first_table.name
    -> );
+------+------+
| id   | name |
+------+------+
|    1 | abhi |
+------+------+
1 row in set (0.01 sec)
```

QUESTION 18 - MINUS

SCENARIO:
Find names that exist only in first_table.

QUERY:

```
SELECT name FROM first_table
MINUS
SELECT name FROM second_table;
```

```
mysql> SELECT name
    -> FROM first_table
    -> WHERE name NOT IN (
    ->     SELECT name
    ->     FROM second_table
    -> );
+------+
| name |
+------+
| abhi |
+------+
1 row in set (0.02 sec)
```

QUESTION 19

SCENARIO:
Find students having matching addresses.

QUERY:

```
SELECT c.id,c.name,ci.address
FROM class c
INNER JOIN class_info ci
ON c.id=ci.id;
```

```
mysql> SELECT c.id, c.name, ci.address
    -> FROM class c
    -> INNER JOIN class_info ci
    -> ON c.id = ci.id;
+------+--------+---------+
| id   | name   | address |
+------+--------+---------+
|    1 | Ananya | DELHI   |
|    1 | Ananya | DELHI   |
|    2 | Akhil  | MUMBAI  |
|    2 | Akhil  | MUMBAI  |
|    1 | Ananya | DELHI   |
|    1 | Ananya | DELHI   |
|    2 | Akhil  | MUMBAI  |
|    2 | Akhil  | MUMBAI  |
+------+--------+---------+
8 rows in set (0.02 sec)
```

QUESTION 20

SCENARIO:
Generate a report showing all students and address availability status.

QUERY:

```
SELECT c.id,
       c.name,
       CASE
           WHEN ci.address IS NULL
           THEN 'Address Missing'
           ELSE 'Address Available'
       END AS Status
FROM class c
LEFT JOIN class_info ci
ON c.id=ci.id;
```

```
mysql> SELECT c.id,
    ->        c.name,
    ->        CASE
    ->            WHEN ci.address IS NULL
    ->            THEN 'Address Missing'
    ->            ELSE 'Address Available'
    ->        END AS Status
    -> FROM class c
    -> LEFT JOIN class_info ci
    -> ON c.id = ci.id;
+------+--------+------------------+
| id   | name   | Status           |
+------+--------+------------------+
|    1 | Ananya | Address Available |
|    1 | Ananya | Address Available |
|    2 | Akhil  | Address Available |
|    2 | Akhil  | Address Available |
|    4 | Bala   | Address Missing   |
|    1 | Ananya | Address Available |
|    1 | Ananya | Address Available |
|    2 | Akhil  | Address Available |
|    2 | Akhil  | Address Available |
|    4 | Bala   | Address Missing   |
|    5 | ashish | Address Missing   |
+------+--------+------------------+
11 rows in set (0.01 sec)
```